Query Processing

Chapter 7

Query Processing

Query Processing is expressed in a high level query language such as SQL.

This query must be scanned, parsed and validated before execution.

The scanner identifies the language tokens such as SQL keywords, attributes name, relation name etc.

The parser check the query syntax to determine whether it is formulated according to the syntax rule of query language.

The query must be validating by checking that all attributes and relation names are valid and meaningful according to the schema of the particular database.

- ✓ Query processing is the procedure of selecting the best plan or strategy to be used in responding to a database request.
- ✓ The plan is then executed to generate a response.
- ✓ The component of the DBMS responsible for generating this strategy is called a query processor. It is a stepwise process.
- ✓ The first step is to transform the query into a standard from.
- ✓ For example, a query in QBE looked into by the research community. **Query** may be transformed into a relational algebraic expression; the optimization is performed by substituting equivalent expressions for those in the query.
- ✓ In the next step a number of strategies called access pans are generated for evaluating the transformed query.
- ✓ The physical characteristics of the data and any supporting access method are taken into account in generating alternate access pans.
- ✓ The cost of each access plan is estimated and the optimal one is chosen and executed.

Objective of Query Processing

The aim of query processing is to transform a query written in a high level language into a correct and efficient execution strategy expressed in a low level language and execute the strategy to retrieve the data.

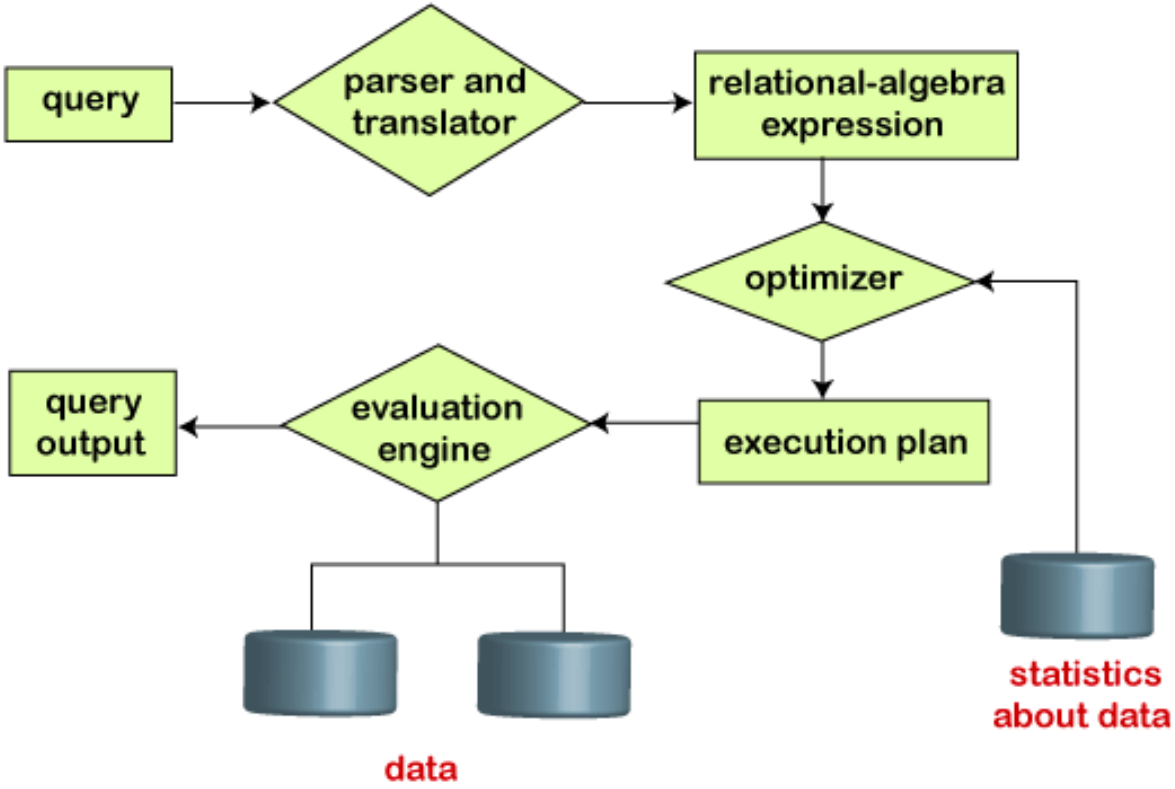
Basic Steps in Query Processing

Query Processing is the activity performed in extracting data from the database.

In query processing, it takes various steps for fetching the data from the database. The steps involved are:

1. Parsing and translation
2. Optimization
3. Evaluation

Basic Steps in Query Processing



Steps in query processing

1. Parsing and Translation

As query processing includes certain activities for data retrieval. Initially, the given user queries get translated in high-level database languages such as SQL.

It gets translated into expressions that can be further used at the physical level of the file system.

After this, the actual evaluation of the queries and a variety of query - optimizing transformations and takes place.

Thus before processing a query, a computer system needs to translate the query into a human-readable and understandable language.

1. Parsing and Translation

Consequently, SQL or Structured Query Language is the best suitable choice for humans. But, it is not perfectly suitable for the internal representation of the query to the system.

Relational algebra is well suited for the internal representation of a query.

The translation process in query processing is similar to the parser of a query.

When a user executes any query, for generating the internal form of the query, the parser in the system checks the syntax of the query, verifies the name of the relation in the database, the tuple, and finally the required attribute value.

The parser creates a tree of the query, known as 'parse-tree.' Further, translate it into the form of relational algebra. With this, it evenly replaces all the use of the views when used in the query.

2. Evaluation

For this, with addition to the relational algebra translation, it is required to annotate the translated relational algebra expression with the instructions used for specifying and evaluating each operation. Thus, after translating the user query, the system executes a query evaluation plan.

Query Evaluation Plan

- In order to fully evaluate a query, the system needs to construct a query evaluation plan.
- The annotations in the evaluation plan may refer to the algorithms to be used for the particular index or the specific operations.
- Such relational algebra with annotations is referred to as **Evaluation Primitives**. The evaluation primitives carry the instructions needed for the evaluation of the operation.
- Thus, a query evaluation plan defines a sequence of primitive operations used for evaluating a query. The query evaluation plan is also referred to as **the query execution plan**.
- A **query execution engine** is responsible for generating the output of the given query. It takes the query execution plan, executes it, and finally makes the output for the user query.

3. Optimization

- The cost of the query evaluation can vary for different types of queries. Although the system is responsible for constructing the evaluation plan, the user does need not to write their query efficiently.
- Usually, a database system generates an efficient query evaluation plan, which minimizes its cost. This type of task performed by the database system and is known as Query Optimization.
- For optimizing a query, the query optimizer should have an estimated cost analysis of each operation. It is because the overall operation cost depends on the memory allocations to several operations, execution costs, and so on.

Finally, after selecting an evaluation plan, the system evaluates the query and produces the output of the query.

Suppose a user executes a query. As we have learned that there are various methods of extracting the data from the database. In SQL, a user wants to fetch the records of the employees whose salary is greater than or equal to 10000. For doing this, the following query is undertaken:

select emp_name from Employee where salary>10000;

Thus, to make the system understand the user query, it needs to be translated in the form of relational algebra. We can bring this query in the relational algebra form as:

- $\sigma_{salary>10000}\ (\pi_{salary}\ (Employee))$
- $\pi_{salary}\ (\sigma_{salary>10000}\ (Employee))$

After translating the given query, we can execute each relational algebra operation by using different algorithms. So, in this way, a query processing begins its working.

- A relational algebra expression may have many equivalent expressions

E.g., $\sigma_{balance<2500}(\Pi_{balance}(account))$ is equivalent to

$$\Pi_{balance}(\sigma_{balance<2500}(account))$$

- Each relational algebra operation can be evaluated using one of several different algorithms
  - Correspondingly, a relational-algebra expression can be evaluated in many ways.
- Annotated expression specifying detailed evaluation strategy is called an evaluation-plan.
  - E.g., can use an index on *balance* to find accounts with balance < 2500,
  - or perform complete relation scan and discard accounts with balance ≥ 2500

The Query execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.

Query Optimization: Amongst all equivalent evaluation plans choose the one with lowest cost.

- Cost is estimated using statistical information from the database catalog e.g. number of tuples in each relation, size of tuples, etc

Alternative ways of evaluating a given query

- Equivalent expressions
- Different algorithms for each operation

Query Tree

Query tree: A tree data structure that corresponds to a relational algebra expression

- It represents the input relations of the query as leaf nodes of the tree, and
- Represents the relational algebra operations as internal nodes

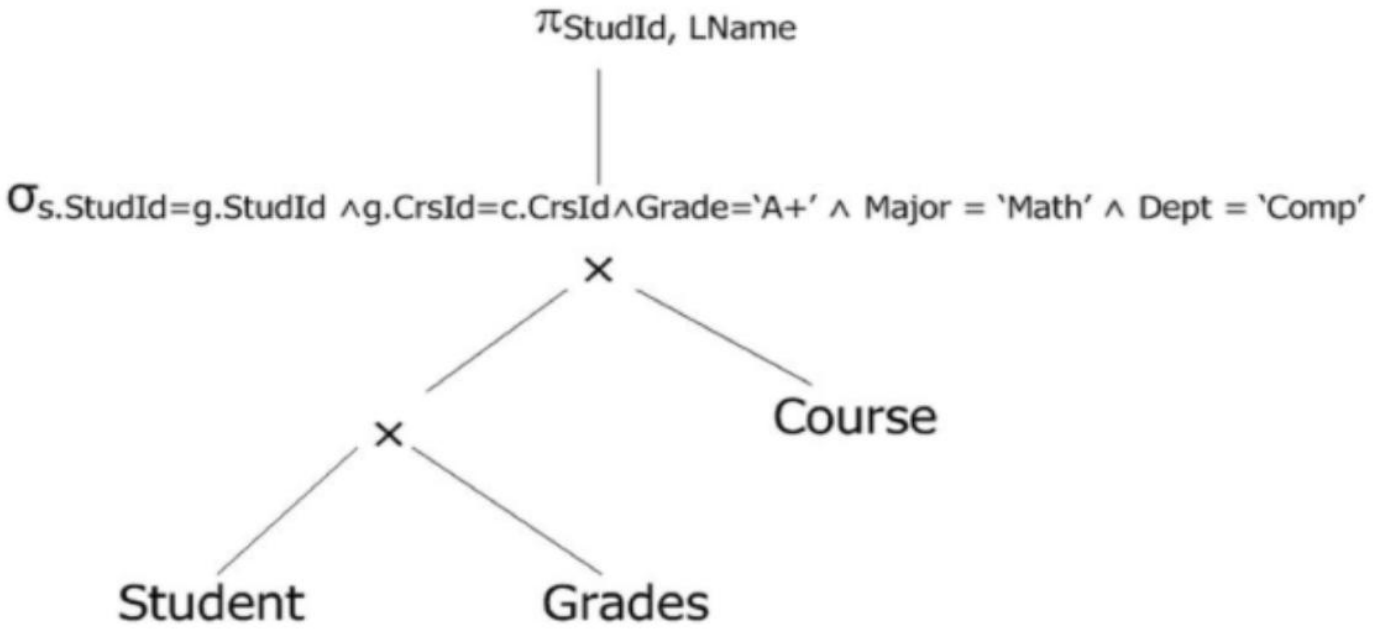
The initial query tree for the example SQL query:

- Lower level nodes, starting from leaves, contain Cartesian product operators
- These are applied onto relations from the SQL FROM clause
- After that are (relational) select and join conditions from SQL WHERE clause to upper tree nodes applied
- Finally is project operator of the SELECT clause attribute list to tree root applied

Query Tree Example

Relational Algebra:

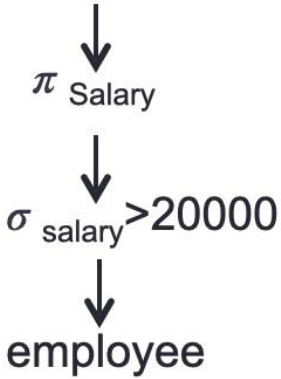
$$\pi_{\text{StudId, LName}} (\sigma_{\text{s.StudId=g.StudId} \wedge \text{g.CrsId=c.CrsId} \wedge \text{Grade='A+'} \wedge \text{Major='Math'} \wedge \text{Dept='Comp'}} (\text{Course} \times (\text{Student} \times \text{Grades})))$$



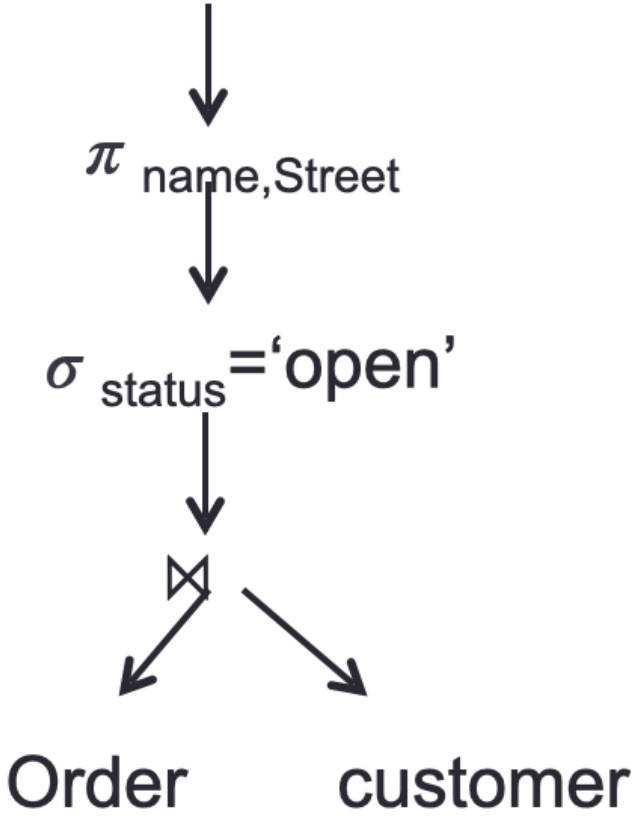
Construction of parse tree

- A leaf node is created for each base relation in query
- A non leaf node is created for each intermediated relation produced by relational algebra expression
- The roof node represents result of query
- The sequence of operation directed from leaves to root

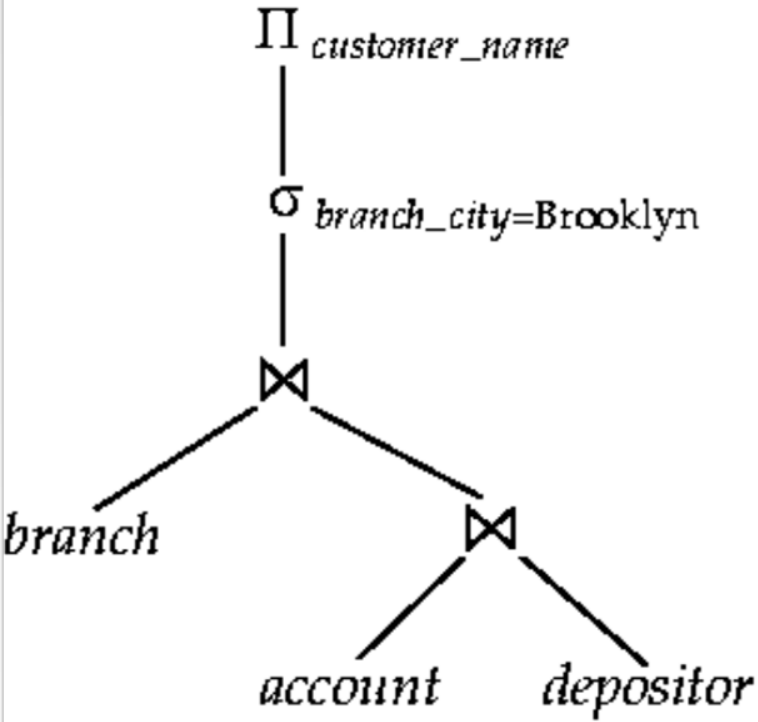
i) $\pi_{\text{Salary}}(\sigma_{\text{salary}}>20000(\text{employee}))$

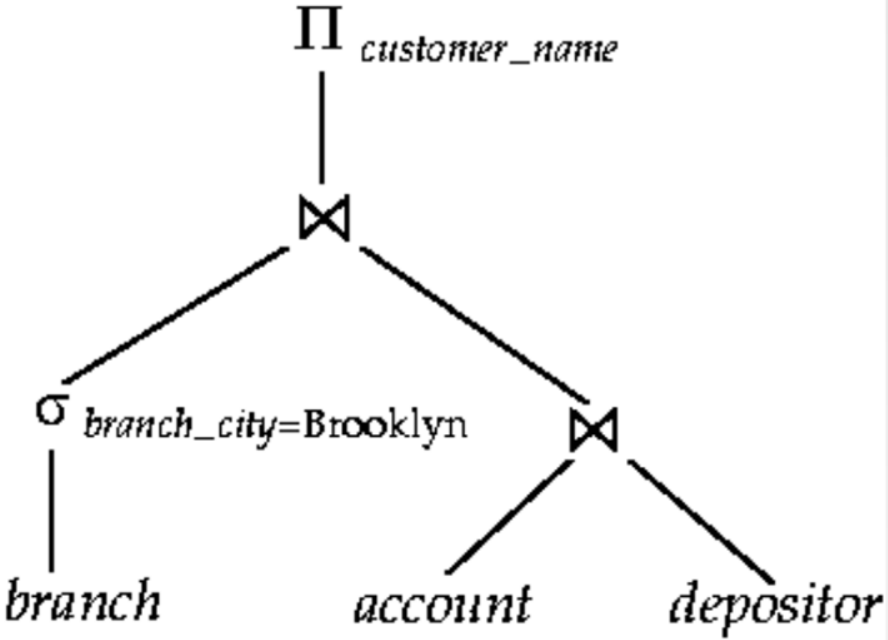


ii) π name, street (σ_{status}='open'(order $\bowtie$ customer))



- $\pi_{customer_name}$ (π_{salary} (($\sigma_{branch_city=\text{"Brooklyn"}}$ (branch)⋈(account ⋈ depositor))
- $\pi_{customer_name}$ ($\sigma_{branch_city=\text{"Brooklyn"}}$ (branch ⋈ account ⋈ depositor))





Generating Equivalent Expressions

Transformation of Relational Expressions

Two relational algebra expressions are said to be equivalent if the two expressions generate the same set of tuples on every legal database instance

- Note: order of tuples is irrelevant

In SQL, inputs and outputs are multisets of tuples

- Two expressions in the multiset version of the relational algebra are said to be equivalent if the two expressions generate the same multiset of tuples on every legal database instance.

An equivalence rule says that expressions of two forms are equivalent

- Can replace expression of first form by second, or vice versa

Equivalence Rules

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. Selection operations are commutative.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\ldots(\Pi_{Ln}(E))\ldots)) = \Pi_{L_1}(E)$$

1. Selections can be combined with Cartesian products and theta joins.

α. $\sigma_{\theta}(E_1 X E_2) = E_1 \bowtie_{\theta} E_2$

β. $\sigma_{\theta 1}(E_1 \bowtie_{\theta 2} E_2) = E_1 \bowtie_{\theta 1 \wedge \theta 2} E_2$

Equivalence Rules (Cont.)

5. Theta-join operations (and natural joins) are commutative.

$$E_1 \bowtie_\theta E_2 = E_2 \bowtie_\theta E_1$$

6. (a) Natural join operations are associative:

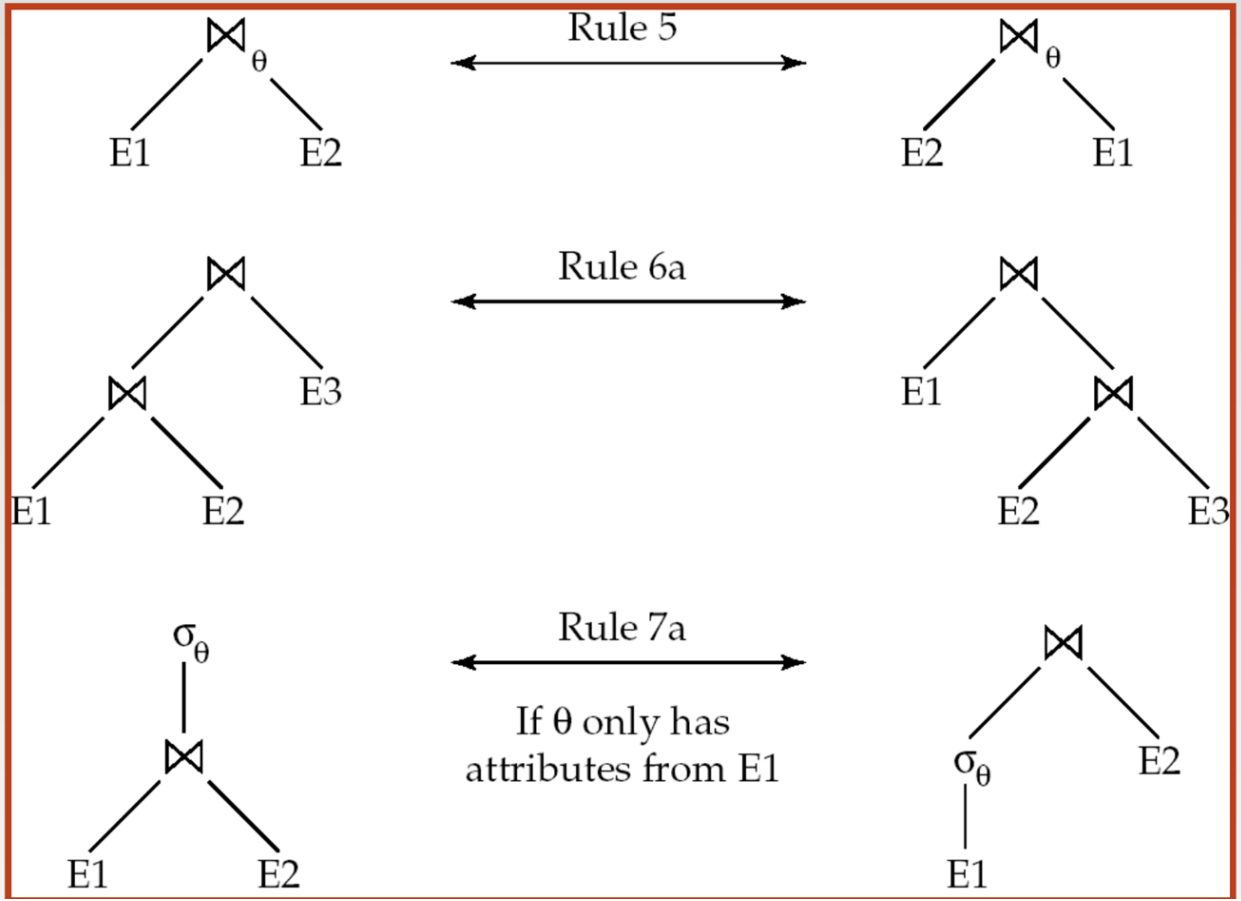
$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

(b) Theta joins are associative in the following manner:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3.

Pictorial Depiction of Equivalence Rules



Equivalence Rules (Cont.)

7. The selection operation distributes over the theta join operation under the following two conditions:
   (a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_\theta E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_\theta E_2$$

   (b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2.

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_\theta E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_\theta (\sigma_{\theta_2}(E_2))$$

Equivalence Rules (Cont.)

8. The projection operation distributes over the theta join operation as follows:

   (a) if θ involves only attributes from $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = (\Pi_{L_1}(E_1)) \bowtie_\theta (\Pi_{L_2}(E_2))$$

   (b) Consider a join $E_1 \bowtie_\theta E_2$.

   - Let L_1 and L_2 be sets of attributes from E_1 and E_2, respectively.
   - Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$, and
   - let L_4 be attributes of E_2 that are involved in join condition θ , but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_\theta (\Pi_{L_2 \cup L_4}(E_2)))$$

Equivalence Rules (Cont.)

1. The set operations union and intersection are commutative

$$E_1 \cup E_2 = E_2 \cup E_1$$
$$E_1 \cap E_2 = E_2 \cap E_1$$

   - (set difference is not commutative).

2. Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$
$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

1. The selection operation distributes over $\cup$, $\cap$ and –.

$$\sigma_\theta (E_1 - E_2) = \sigma_\theta (E_1) - \sigma_\theta (E_2)$$

and similarly for $\cup$ and $\cap$ in place of –

Also: $\sigma_\theta (E_1 - E_2) = \sigma_\theta (E_1) - E_2$

and similarly for $\cap$ in place of –, but not for $\cup$

12. The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$